

Resúmenes

Modulo 2: Fundamentos de python 4

• La función Print()

Una función es código capaz de causar un efecto y evaluar un valor. Puede venir de Python integrada o escrita por el programador. Print es integrada e imprime en la terminal. Toda función necesita parentesis, aunque no lleven argumentos. Los argumentos van dentro de ellos, acepta casi cualquier tipo de argumento, envia datos a la consola y no evalua ningun valor.

Escape y nueva linea: \ es el caracter de caracter; \n inicia una nueva linea.

Argumentos multiples: Separados por comas, los junta en una misma linea.

• Literales

Una literal es un valor fijo escrito directamente en el código fuente. A diferencia de las variables, su valor no cambia durante la ejecución del programa.

Enteros: Numeros sin decimales. Se puede usar guiones bajos para facilitar la lectura.

Octales y hexadecimal: Sistema numericos alternativos. Usan el prefijo 0o o 0x.

Flotantes: Numeros con punto decimal. Python permite omitir los ceros a los extremos.

• Operadores y Expresiones

** Exponenciación: El operador izquierdo es la base y el derecho el exponente.

* Multiplicación: Sigue la misma regla si los numeros son enteros el resultado es entero, mientras que si es flotante el resultado es flotante.

/ División: Siempre devuelve un numero flotante, incluso si la división es exacta.

// División entera: Divide y redondea el resultado siempre hacia abajo al entero mas cercano.

% Modulo: Devuelve el residuo de la división entera.

• Variable:

Es una etiqueta o nombre que se le asigna a un valor para guardarlo y usarlo después en la memoria. Creación Se crea al asignarle un valor con =

• Comentarios.

Un comentario es un texto ignorado por Python al ejecutarse; comienza con un # y llega hasta el final de la línea. Se usa para explicar el código o para desactivar temporalmente fragmentos del código ("comentar / descomentar"), útil al depurar.

Un buen nombre de una variable puede ser auto-comentable.

• función input.

A diferencia de print() que es el que envía datos, input() obtiene datos del usuario y los pasa al programa: su resultado debe ser asignado a una variable o si no se pierde. Invocada sin argumentos, pone la consola en modo entrada. Invocada con un argumento (una cadena), muestra ese mensaje antes de esperar la entrada.

El resultado del input es siempre una cadena, nunca un número, por lo que no se puede usar directamente en operaciones matemáticas ya que se generaría un TypeError.

Para resolver eso existen las funciones int() y float(), que convierten una cadena a entero o flotante respectivamente.

• Operadores de cadena.

Los operadores, + y * tienen un uso adicional al aplicarse sobre cadenas.

Concatenación: El + aplicado a dos cadenas actúa como concatenador, siempre y cuando ambas sean cadenas.

Replicación: El * aplicado a una cadena y un número la replica ese número de veces.

Exámenes

Python INSTITUTE

« PE1 – Module 2 Quiz »

Progreso (100%)

Tu puntaje: 10/10

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 2 Quiz

100%

Volver a presentar la prueba

Revisar prueba

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

« FP1: Examen Módulo 2 »

Progreso (100%)

Tu puntaje: 20/20

100%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 2 Test

100%

Volver a presentar la prueba

Revisar prueba

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Laboratorios

Python INSTITUTE

2.1.1.6 LABORATORIO: La función print()

MODULE (10%)

SECTION (33%)

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Muy fácil

Objetivos

- Familiarizarse con la función `print()` y sus capacidades de formato.
- Experimentar con el código de Python.

Escenario

El comando `print()`, el cual es una de las directivas más sencillas de Python, simplemente imprime una línea de texto en la pantalla.

En tu primer laboratorio:

- Utiliza la función `print()` para imprimir la línea `"¡Hola, Mundo!"` en la pantalla.
- Una vez hecho esto, utiliza la función `print()` nuevamente, pero esta vez imprime tu nombre.
- Elimina las comillas dobles y ejecuta el código. Observa la reacción de Python. ¿Qué tipo de error se produce?
- Luego, elimina los paréntesis, vuelve a poner las comillas dobles y vuelve a ejecutar el código. ¿Qué tipo de error se produce esta vez?
- Experimenta tanto como puedas. Cambia las comillas dobles a comillas simples, utiliza múltiples funciones `print()` en la misma línea y luego en líneas

```
1 # Jairo Haziel Rosas Enriquez
2
3 print ("¡Hola, Mundo!")
4 print ("Jairo Rosas")
5
6 print ('¡Hola, Mundo!')
7 print ('Jairo Rosas')
```

Console >_

```
¡Hola, Mundo!
Jairo Rosas
¡Hola, Mundo!
Jairo Rosas
```

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go...

Edube Interactive - 2.2.1.11 LAI...

Pregúntale a Gemini

edube.org/learn/python-essentials-1-esp-edube/laboratorio-literales-de-python-cadenas-3

Python INSTITUTE

« 2.2.1.11 LABORATORIO: Literales de Python - Cadenas »

MODULE (44%)

SECTION (92%)

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con la función `print()` y sus capacidades de formato.
- Practicar el codificar cadenas.
- Experimentar con el código de Python.

Escenario

Escribe una sola línea de código, utilizando la función `print()`, así como los caracteres de nueva línea y escape, para obtener la salida esperada de tres líneas.

Salida Esperada

```
"Estoy"
"aprendiendo"
"python"
```

```
1 print("\n\"Estoy\\n\\n\"aprendiendo\\n\\n\\n\"Python\\n\\n\")
```

Console >_

```
"Estoy"
"aprendiendo"
"python"
```

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go...

Edube Interactive - 2.4.1.7 LAB...

Pregúntale a Gemini

edube.org/learn/python-essentials-1-esp-edube/laboratorio-variables-2

Python INSTITUTE

« 2.4.1.7 LABORATORIO: Variables »

MODULE (69%)

SECTION (64%)

LABORATORIO

Tiempo Estimado

10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con el concepto de almacenar y trabajar con diferentes tipos de datos en Python.
- Experimentar con el código en Python.

Escenario

A continuación una historia:

Érase una vez en la Tierra de las Manzanas, Juan tenía tres manzanas, María tenía cinco manzanas, y Adán tenía seis manzanas. Todos eran muy felices y vivieron por muchísimo tiempo. Fin de la Historia.

Tu tarea es:

- Crear las variables: `juan`, `maria` y `adan`.
- Asignar valores a las variables. El valor debe de ser igual al número de manzanas que cada quien tenía.
- Una vez almacenados los números en las variables, imprimir las variables en una línea, y separar cada una de ellas con una coma.
- Después se debe crear una nueva variable llamada `total_manzanas` y se debe igualar a la suma de las tres variables anteriores.
- Imprime el valor almacenado en `total_manzanas` en la consola.
- Experimenta con tu código: crea nuevas variables, asigna diferentes valores a ellas, y realiza varias operaciones aritméticas con ellas (por ejemplo, `+`, `-`, `*`, `/`, `//`, etc.). Intenta poner una cadena con un entero juntos en la misma línea, por ejemplo: `"Número Total de Manzanas:"` y `total_manzanas`.

```
1 juan = 3
2 maria = 5
3 adan = 6
4
5 print(juan, maria, adan, sep=", ")
6
7 total_manzanas = juan + maria + adan
8 print(total_manzanas)
```

Console >_

```
3, 5, 6
14
```

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTO

2.4.1.9 LABORATORIO: Variables, un sencillo convertidor

Escenario

Millas y kilómetros son unidades de longitud o distancia.

Teniendo en mente que 1 milla equivale aproximadamente a 1.61 kilómetros, complementa el programa en el editor para que convierta de:

- Millas a kilómetros.
- Kilómetros a millas.

No se debe cambiar el código existente. Escribe tu código en los lugares indicados con `###`. Prueba tu programa con los datos que han sido provistos en el código fuente.

Pon mucha atención a lo que esta ocurriendo dentro de la función `print()`. Analiza como es que se proveen múltiples argumentos para la función, y como es que se muestra el resultado.

Nota que algunos de los argumentos dentro de la función `print()` son cadenas (por ejemplo `"millas son"`) y otros son variables (por ejemplo `millas`).

Tip

Hay una cosa interesante más que esta ocurriendo. ¿Puedes ver otra función dentro de la función `print()`? Es la función `round()`. Su trabajo es redondear la salida del resultado al número de decimales especificados en el paréntesis, y regresar un valor flotante (dentro de la función `round()` se puede encontrar el nombre de la variable, el nombre, una coma, y el número de decimales que se desean mostrar). Se hablará más de esta función muy pronto, no te preocupes si no todo queda muy claro. Solo se quiere impulsar tu curiosidad.

Después de completar el laboratorio, abre Sandbox, y experimenta más. Intenta escribir diferentes convertidores, por ejemplo, un convertidor de USD a EUR, un convertidor de temperatura, etc. ¡Deja que tu imaginación vuele! Intenta mostrar los resultados combinando cadenas y variables. Intenta utilizar y experimentar con la función `round()` para redondear tus resultados a uno, dos o tres decimales. Revisa que es lo que sucede si no se provee un dígito al redondear. Recuerda probar tus programas.

Experimenta, saca tus propias conclusiones, y aprende. Sé curioso.

Resultado Esperado

```
7.38 millas son 11.88 kilómetros
12.25 kilómetros son 7.61 millas
```

```
1 kilometers = 12.25
2 millas = 7.38
3
4 millas_to_kilometers = millas * 1.61
5 kilometers_to_millas = kilometers / 1.61
6
7 print(millas, "millas son", round(millas_to_kilometers, 2), "kilómetros")
8 print(kilometers, "kilómetros son", round(kilometers_to_millas, 2), "millas")
```

```
Console >_
7.38 millas son 11.88 kilómetros
12.25 kilómetros son 7.61 millas
```

Python INSTITUTO

2.4.1.10 LABORATORIO: Operadores y expresiones

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con los conceptos de números, operadores y operaciones aritméticas en Python.
- Realizar cálculos básicos.

Escenario

Observa el código en el editor: lee un valor `flotante`, lo coloca en una variable llamada `x`, e imprime el valor de la variable llamada `y`. Tu tarea es completar el código para evaluar la siguiente expresión:

$$3x^3 - 2x^2 + 3x - 1$$

El resultado debe ser asignado a `y`.

Recuerda que la notación algebraica clásica muy seguido omite el operador de multiplicación, aquí se debe de incluir de manera explícita. Nota como se cambia el tipo de dato para asegurarnos de que `x` es del tipo `flotante`.

Mantén tu código limpio y legible, y pruébalo utilizando los datos que han sido proporcionados. No te desanimes por no lograrlo en el primer intento. Se persistente y curioso.

Datos de Prueba

Entrada de Muestra

```
x = 0
x = 1
x = -1
```

Salida Esperada

```
y = -1.0
y = 3.0
y = -9.0
```

```
1 x = 0
2 x = float(x)
3 y = 3 * x**3 - 2 * x**2 + 3 * x - 1
4 print("y =", y)
5
```

```
Console >_
y = -1.0
```

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go...Edube Interactive - 2.5.1.2 LAB...+Pregúntale a Gemini

edube.org/learn/python-essentials-1-esp-edube/laboratorio-comentarios-2

PythonINSTITUTE

2.5.1.2 LABORATORIO: Comentarios

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de Dificultad

Muy Fácil

Objetivos

- Familiarizarse con el concepto de comentarios en Python.
- Utilizar y no utilizar los comentarios.
- Reemplazar los comentarios con código.
- Experimentar con el código de Python.

Escenario

El código en el editor contiene comentarios. Intenta mejorarlo: agrega o quita comentarios donde consideres que sea apropiado (en ocasiones el remover un comentario lo hace mas legible). además, cambia el nombre de las variables donde consideres que esto mejorará la comprensión del código.

NOTA

Los comentarios son muy importantes. No solo hacen que el programa sea **más fácil de entender**, pero también sirven para **deshabilitar aquellas partes de código que no son necesarias** (por ejemplo, cuando se necesita probar cierta parte del código, e ignorar el resto). Los buenos programadores **describen** cada parte importante del código, y dan **nombres significativos** a variables, debido a que en ocasiones es mucho más sencillo dejar el comentario dentro del código mismo.

Es bueno utilizar nombres de variables **legibles**, y en ocasiones es mejor **dividir el código** en partes con nombres (por ejemplo en funciones). En algunas situaciones, es una buena idea escribir los pasos de como se realizaron los cálculos de una forma sencilla y clara.

Una cosa mas: puede ocurrir que un comentario contenga una pieza de información incorrecta o errónea, nunca se debe de hacer eso a propósito.

```
1 horas = 2
2 segundos_por_hora = 3600
3
4 print("Horas: ", horas)
5 print("Segundos en Horas: ", horas * segundos_por_hora)
6 print("Adiós")
7
8
```

Console >_

Horas: 2
Segundos en Horas: 7200
Adiós

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go...Edube Interactive - 2.6.1.9 LAB...+Pregúntale a Gemini

edube.org/learn/python-essentials-1-esp-edube/laboratorio-entradas-y-salidas-simples-2

PythonINSTITUTE

2.6.1.9 LABORATORIO: Entradas y salidas simples

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con la entrada y salida de datos en Python.
- Evaluar expresiones simples.

Escenario

La tarea es completar el código para evaluar y mostrar el resultado de cuatro operaciones aritméticas básicas.

El resultado debe ser mostrado en consola.

Quizá no podrás proteger el código de un usuario que intente dividir entre cero. Por ahora, no hay que preocuparse por ello.

Prueba tu código - ¿Produce los resultados esperados?

No te mostraremos ningún dato de prueba, eso sería demasiado sencillo.

```
1 # Ingresa un valor flotante para la variable a aqui
2 a = float(input("Inserta el valor para a: "))
3 # Ingresa un valor flotante para la variable b aqui
4 b = float(input("Inserta el valor para b: "))
5
6 # Muestra el resultado de la suma aqui
7 print("Resultado de la suma: ", a + b)
8 # Muestra el resultado de la resta aqui
9 print("Resultado de la resta: ", a - b)
10 # Muestra el resultado de la multiplicación aqui
11 print("Resultado de la multiplicación: ", a * b)
12 # Muestra el resultado de la división aqui
13 print("Resultado de la división: ", a / b)
14
15 print("\n¡Eso es todo, amigos!")
```

Console >_

Inserta el valor para a: 20
Inserta el valor para b: 21
Resultado de la suma: 46.0
Resultado de la resta: 4.0
Resultado de la multiplicación: 420.0
Resultado de la división: 1.1904761904761905

¡Eso es todo, amigos!

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

2.6.1.10 LABORATORIO: Operadores y expresiones

MODULE (93%)

SECTION (77%)

Comprender la precedencia y asociatividad de los operadores de Python, así como el correcto uso de los paréntesis.

Escenario

La tarea es completar el código para poder evaluar la siguiente expresión:

$$\frac{1}{x + \frac{1}{x + \frac{1}{x + \frac{1}{x}}}}$$

El resultado debe de ser asignado a `y`. Se cauteloso, observa los operadores y priorízalos. Utiliza cuantos paréntesis sean necesarios.

Puedes utilizar variables adicionales para acortar la expresión (sin embargo, no es muy necesario). Prueba tu código cuidadosamente.

Datos de Prueba

Entrada de muestra: `1`

Salida esperada:

```
y = 0.6000000000000001
```

Entrada de muestra: `1.5`

```
1 x = float(input("Ingresa el valor para x: "))
2
3 y = 1 / (x + 1 / (x + 1 / (x + 1 / x)))
4
5 print("y =", y)
```

Console >_

Ingresa el valor para x: 1
y = 0.6000000000000001

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

2.6.1.11 LABORATORIO: Operadores y expresiones

MODULE (94%)

SECTION (85%)

La tarea es preparar un código simple para evaluar o encontrar el tiempo final de un periodo de tiempo dado, expresándolo en horas y minutos. Las horas van de 0 a 23 y los minutos de 0 a 59. El resultado debe ser mostrado en la consola.

Por ejemplo, si el evento comienza a las **12:17** y dura **59 minutos**, terminará a las **13:16**.

No te preocupes si tu código no es perfecto, está bien si acepta una hora inválida, lo más importante es que el código produzca una salida correcta acorde a la entrada dada.

Prueba el código cuidadosamente. Pista: utilizar el operador `%` puede ser clave para el éxito.

Datos de Prueba

Entrada de muestra:

```
12
17
59
```

Salida esperada: `13:16`

Entrada de muestra:

```
23
58
642
```

Salida esperada: `10:40`

Entrada de muestra:

```
1 hour = int(input("Hora de inicio (horas): "))
2 mins = int(input("Minuto de inicio (minutos): "))
3 dura = int(input("Duración del evento (minutos): "))
4
5 # Sumamos los minutos iniciales con la duración del evento
6 total_mins = mins + dura
7
8 # Calculamos las horas adicionales que se acumulan y los minutos extra
9 extra_hours = total_mins // 60
10 final_mins = total_mins % 60
11
12 # Calculamos la hora final utilizando el operador módulo para horas
13 final_hour = (hour + extra_hours) % 24
14
15 # Imprimimos el resultado final
16 print(final_hour, ":", final_mins, sep=":")
```

Console >_

Hora de inicio (horas): 12
Minuto de inicio (minutos): 17
Duración del evento (minutos): 59
13:16

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome